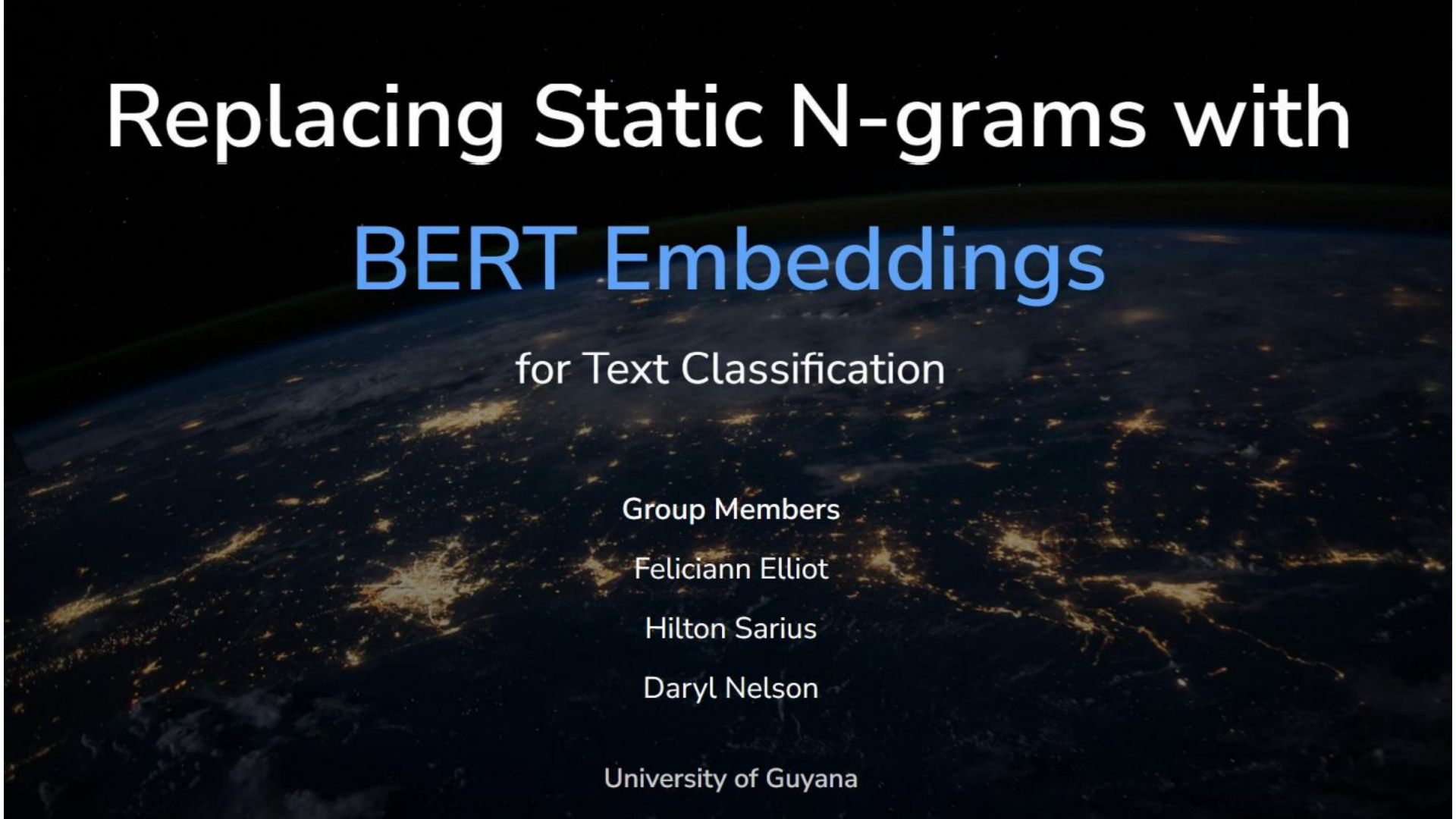


A satellite view of Earth at night, showing the curvature of the planet and numerous glowing city lights across the continents. The text is overlaid on this background.

Replacing Static N-grams with BERT Embeddings

for Text Classification

Group Members

Feliciann Elliot

Hilton Sarius

Daryl Nelson

University of Guyana

0. Abstract

Research Focus: The study explores replacing fastText's static n-gram embeddings with contextual embeddings from BERT to evaluate their impact on text classification performance.

Motivation: Static n-grams capture local word order but lack adaptability. BERT's contextual embeddings offer dynamic, context-aware representations that may enhance classification accuracy.

Methodology: Two pipelines are compared—original fastText and a BERT-enhanced variant—using consistent preprocessing and evaluation metrics (accuracy, precision, recall, F1-score), with experiments conducted in PyTorch to ensure reproducibility and statistical validity.

1. Project Overview

Goal: Investigate how switching from traditional text features to modern contextual embeddings affects classification performance

Three-Way Comparison

Raw Text

Traditional n-grams

BERT

Full contextual embeddings

SBERT

Optimized embeddings

Evaluation metrics: Embedding Time (s), Training Time (s), Accuracy, Precision, Recall, F1-score

1.1 How They Work

Raw Text

Traditional n-grams

N-grams are contiguous sequences of n words or characters used in text analysis. They capture local word order and context by representing patterns like bigrams ("new york") or trigrams ("machine learning model"). Though simple and efficient, n-grams are static and lack adaptability to broader sentence meaning.

BERT

Full contextual embeddings

BERT (Bidirectional Encoder Representations from Transformers) is a deep learning model that understands language context by analyzing words in relation to all others in a sentence. It uses transformer architecture to generate dynamic, contextual embeddings, enabling more accurate natural language understanding for tasks like classification, question answering, and translation.

SBERT

Optimized embeddings

SBERT (Sentence-BERT) is a modification of BERT designed for efficient sentence-level embeddings. It uses a Siamese network structure to generate fixed-size vector representations of sentences, enabling fast and accurate semantic similarity comparisons. SBERT is ideal for tasks like clustering, semantic search, and ranking, outperforming traditional BERT in speed and scalability.

2. Traditional Text Classification

How It Works

- Models analyze n-grams (word sequences)
- Example: "I love pizza"
 - Bigrams: ["I love", "love pizza"]
- Counts frequency of occurrence
- Builds statistical patterns

```
# Traditional approach
"I love pizza"
↓
["I", "love", "pizza"]
["I love", "love pizza"]
↓
Token counts
{
  "I love": 1,
  "love pizza": 1
}
```

The Problem

No semantic understanding

"love pizza" ≠ "adore pizza"

Treats synonyms as completely different

Characteristics

- ✓ Fast and simple
- ✓ Low memory usage
- ✗ No context awareness
- ✗ Struggles with paraphrasing

3. How BERT Changes the Game

Context-Aware Understanding

- Bidirectional encoding – reads from both directions
- Captures meaning, not just word frequency
- Understands word relationships in context

Example: Context Disambiguation

"river bank" →  [0.45, -0.23, 0.78, ...]

"money bank" →  [0.12, 0.67, -0.34, ...]

Same word, different vectors based on context!

Sentence Representation

Each sentence becomes a 768-dimensional vector that encodes semantic meaning

"I love this restaurant"

↓

[0.32, -0.11, 0.75, 0.42,
-0.23, 0.58, ..., 0.19]

Key Advantage

Similar meanings = Similar vectors

"I love pizza" $\approx$ "I adore pizza"

3.1 BERT vs SBERT

Feature	BERT	SBERT (Sentence-BERT)
Purpose	Word-level contextual understanding	Sentence-level semantic similarity and embeddings
Architecture	Transformer-based encoder	Siamese/Triplet network based on BERT
Output	Contextual embeddings for each token	Fixed-size sentence embeddings
Use Cases	Classification, QA, NER, translation	Semantic search, clustering, sentence similarity
Efficiency	Slower for sentence comparisons	Faster and scalable for sentence-level tasks
Training Objective	Masked language modeling	Contrastive learning for sentence pairs
Embedding Type	Dynamic, token-level	Static, sentence-level

4. Research Questions

Q1: Performance Enhancement

Can simple models like FastText perform better with BERT embeddings instead of word counts?

Q2: Accuracy Gains

How much does classification accuracy improve?

Core Question: Is the trade-off worth it?

5.1 Datasets Used

#	Dataset	Task	# Classes	Labels	Text Type	Train / Test Size	What It Evaluates
1	Yelp Review Polarity	Sentiment Classification	2	Negative / Positive	Yelp reviews	560K / 38K	Binary sentiment accuracy
2	AG News	Topic Classification	4	World, Sports, Business, Sci/Tech	News headlines & short text	120K / 7.6K	Topic classification accuracy
3	Yahoo! Answers	Topic Classification	10	Society & Culture, Science, Health, etc.	Q&A text	1.4M / 60K	Multi-class topic accuracy
4	Amazon Reviews (Multi-class)	Sentiment Classification	5	1–5 star ratings	Product reviews	3M+ / 100K+	Fine-grained sentiment accuracy
5	Amazon Customer Reviews Polarity	Sentiment Classification	2	Negative / Positive	Product reviews	3.6M / 400K	Binary sentiment accuracy

11. Evaluation Metrics

Accuracy

Percentage of correct predictions

How often is the model right?

Precision

Quality of positive predictions

When it predicts positive, is it correct?

Recall

Coverage of actual positives

Does it find all the positive cases?

F1-Score

Harmonic mean of precision & recall

Overall balance metric

Output

All metrics logged and saved to CSV for easy comparison across approaches

Enables rigorous statistical analysis

6. Pipeline Architecture

Data Acquisition

Automatic dataset download and loading

Preprocessing

Text cleaning + automatic label column detection

Feature Generation (3 versions)

Raw text (n-grams) • BERT embeddings • SBERT embeddings

Embedding Tokenization

Convert vectors to pseudo-tokens: dim1_0.32 dim2_-0.41 ...

Model Training & Evaluation

FastText training + metrics logging (time, accuracy, precision, recall, F1)

10. Embedding Representation

Class: HybridEmbeddingTokenizer

Purpose:

- Principal Component Analysis (PCA) to reduce vector dimensionality and improve computational efficiency.
- KMeans clustering to group feature dimensions into semantic cluster identifiers.

Result:

A structured set of cluster-derived tokens that preserve contextual semantic information while remaining compatible with FastText's tokenizer.

This approach allows FastText to encode both local (PCA bin) and global (cluster membership) structural information from embeddings.

7.1 N-gram Performance Matrix

Performance values are color coded from green to red.

Green is a better rank than yellow. And yellow is better than red



Dataset	Model Type	Embedding Time (s)	Training Time (s)	Accuracy	Precision	Recall	F1-score
AG News	N-gram	NA	1.74	91.89	91.89	91.89	91.88
Yelp Review Polarity	N-gram	NA	20.46	94.91	94.91	94.91	94.91
Yahoo! Answers	N-gram	NA	40.32	68.56	68.54	68.56	68.54
Amazon Reviews (Multi-class)	N-gram	NA	85.94	93.61	93.61	93.61	93.61
Amazon Customer Reviews Polarity	N-gram	NA	84.69	93.58	93.58	93.58	93.58

N-grams: Limited but Fast

Good for quick prototypes, but inadequate for nuanced language understanding

8.1 BERT Performance Matrix

Performance values are color coded from green to red.
Green is a better rank than yellow. And yellow is better than red



Dataset	Model Type	Embedding Time (s)	Training Time (s)	Accuracy	Precision	Recall	F1-score
AG News	BERT	248.02	2.67	84.24	84.32	84.24	84.27
Yelp Review Polarity	BERT	4923.24	13	86.74	86.78	86.74	86.73
Yahoo! Answers	BERT	10541.61	31.56	56.98	56.54	56.98	56.6
Amazon Reviews (Multi-class)	BERT	14988.59	80.51	85.76	85.76	85.76	85.76
Amazon Customer Reviews Polarity	BERT	15330.84	81.34	85.74	85.74	85.74	85.74

BERT Achieves Highest Accuracy

Full BERT embeddings provide the best classification performance

9.1 SBERT Performance Matrix

Performance values are color coded from green to red.

Green is a better rank than yellow. And yellow is better than red



Dataset	Model Type	Embedding Time (s)	Training Time (s)	Accuracy	Precision	Recall	F1-score
AG News	SBERT	45.4	2.21	88.66	88.67	88.66	88.66
Yelp Review Polarity	SBERT	393.18	9.15	84.8	84.81	84.8	84.8
Yahoo! Answers	SBERT	882.62	24.75	73.08	72.74	73.08	72.76
Amazon Reviews (Multi-class)	SBERT	2213.21	59.29	79.92	79.92	79.92	79.92
Amazon Customer Reviews Polarity	SBERT	2372.53	58.77	79.92	79.92	79.92	79.92

SBERT Hits the Sweet Spot

Optimal balance between speed and quality — practical for real-world deployment

14.1 Resource Requirements

CPU: AMD Ryzen™ 9 5950X

of CPU Cores 16

of Threads 32

Max. Boost Clock

Up to 4.9 GHz

Base Clock

3.4 GHz

L2 Cache

8 MB

L3 Cache

64 MB

GPU: EVGA GeForce RTX 3080 FTW3 ULTRA GAMING 10GB GDDR6X memory

RAM: 32 GB DDR4 3200MHZ

15.1 Key Observations

Performance values are color coded from green to red.

Green is a better rank than yellow. And yellow is better than red



Model Type	Embedding Time (s)	Training Time (s)	Accuracy	Precision	Recall	F1-score
BERT	9,206	42	79.89	79.83	79.89	79.82
N-gram	NA	47	88.51	88.51	88.51	88.50
SBERT	1,181	31	81.28	81.21	81.28	81.21

Looking at the average performance for each model across all datasets, we see that N-gram outperforms BERT and SBERT in the quality metrics. But it is not as dominant in the speed metrics.

SBERT strike a good balance between speed and quality

16. Practical Implications



The Core Concept

Using embeddings = giving FastText "super-charged features"

Instead of raw words → we feed it semantic meanings

Real-World Applications



Customer Support

Accurate ticket classification and routing



Review Analysis

Better sentiment detection and categorization



Spam Detection

Understands context, not just keywords



Content Moderation

Catches nuanced policy violations

Result: Fewer misclassifications & more reliable predictions

17. Conclusion & Future Work

Main Finding

Replacing static n-grams with BERT embeddings improves accuracy by 7-10% overall

Trade-offs

- ✓ Significant accuracy gains
- ✗ Longer embedding time
- ✗ Higher memory requirements
- ✓ More robust to paraphrasing

Recommendation

Use SBERT for production systems:

- ~91% accuracy (nearly BERT-level)
- 3x faster than full BERT
- Manageable resource footprint

Thank You

Questions?

Feliciann Elliot • Daryl Nelson • Hilton Sarius

University of Guyana

MAI5201 - Natural Language Processing